Символьный тип данных char

Хоть тип char и относится к **целочисленным типам данных** (и, таким образом, следует всем их правилам), работа с char несколько отличается от работы с обычными целочисленными типами.

Оглавление:

1. **Тип данных char**
2. **Вывод символов**
3. **Оператор static_cast**
4. **Ввод символов**
5. **Размер, диапазон и знак типа char**
6. **Управляющие символы**
7. **Что использовать: '\n' или std::endl?**
8. ***Другие символьные типы: wchar_t, char16_t и char32_t***
9. **В чём разница между одинарными и двойными кавычками при использовании с символами?**

Тип данных char

Переменная типа char занимает 1 байт. Однако вместо конвертации значения типа char в целое число, оно интерпретируется как ASCII-символ.

ASCII (сокр. от *«American Standard Code for Information Interchange»*) — это американский стандартный код для обмена информацией, который определяет способ представления символов английского языка (+ несколько других) в виде чисел от 0 до 127. Например: код буквы '**a**' — 97, код буквы '**b**' — 98. Символы всегда помещаются в одинарные кавычки.

Таблица ASCII-символов:

Код	Символ	Код	Символ	Код	Символ	Код	Символ
0	NUL (null)	32	(space)	64	@	96	`
1	SOH (start of header)	33	!	65	A	97	a
2	STX (start of text)	34	"	66	B	98	b
3	ETX (end of text)	35	#	67	C	99	c
4	EOT (end of transmission)	36	$	68	D	100	d
5	ENQ (enquiry)	37	%	69	E	101	e
6	ACK (acknowledge)	38	&	70	F	102	f
7	BEL (bell)	39	'	71	G	103	g
8	BS (backspace)	40	(	72	H	104	h
9	HT (horizontal tab)	41	)	73	I	105	i
10	LF (line feed/new line)	42	*	74	J	106	j
11	VT (vertical tab)	43	+	75	K	107	k
12	FF (form feed / new page)	44	,	76	L	108	l
13	CR (carriage return)	45	—	77	M	109	m
14	SO (shift out)	46	.	78	N	110	n
15	SI (shift in)	47	/	79	O	111	o
16	DLE (data link escape)	48	0	80	P	112	p
17	DC1 (data control 1)	49	1	81	Q	113	q
18	DC2 (data control 2)	50	2	82	R	114	r
19	DC3 (data control 3)	51	3	83	S	115	s
20	DC4 (data control 4)	52	4	84	T	116	t
21	NAK (negative acknowledge)	53	5	85	U	117	u
22	SYN (synchronous idle)	54	6	86	V	118	v
23	ETB (end of transmission block)	55	7	87	W	119	w
24	CAN (cancel)	56	8	88	X	120	x
25	EM (end of medium)	57	9	89	Y	121	y
26	SUB (substitute)	58	:	90	Z	122	z
27	ESC (escape)	59	;	91	[	123	{
28	FS (file separator)	60	<	92	\	124	|
29	GS (group separator)	61	=	93	]	125	}
30	RS (record separator)	62	>	94	^	126	~
31	US (unit separator)	63	?	95	_	127	DEL (delete)

Символы от 0 до 31 в основном используются для форматирования вывода. Большинство из них уже устарели.

Символы от 32 до 127 используются для кодирования всех букв, цифры, знаки препинания, которые большинство компьютеров используют для отображения текста (на английском языке).

Следующие два **стейтмента** выполняют одно и то же (присваивают переменным типа char целое число 97):

```
char ch1{97};
char ch2{'a'};
```

Будьте внимательны при использовании фактических чисел с числами, которые используются для представления символов (из ASCII-таблицы). Следующие два стейтмента выполняют не одно и то же:

```
char ch{5};
char ch{'5'};
```

Вывод символов

При выводе переменных типа char, **объект cout** выводит символы вместо цифр:

```
#include <iostream>

int main()
{
    char ch{97};
    cout << ch << " " << ch << endl;

    char ch{'5'};
    cout << ch << endl;

    return 0;
}
```

Результат:

```
a 97
5
```

Оператор static_cast

Если вы хотите вывести символы в виде цифр, а не в виде букв, то вам нужно сообщить cout выводить переменные типа char в виде целочисленных значений. Не очень хороший способ это сделать — приказать переменной типа int переменную типа char и вывести её:

```
#include <iostream>

int main()
{
    char ch{97};
    int i{ch};
    cout << i << " " << ch << endl;

    int ch{'5'};
    cout << ch << endl;

    return 0;
}
```

Результат:

```
97
5
```

Лучшим способом является конвертация переменной из одного типа данных в другой с помощью **оператора static_cast**.

Синтаксис static_cast выглядит следующим образом:

```
static_cast(новый_тип_данных)(выражение)
```

Оператор static_cast принимает значение из (**выражения**) в качестве входных данных и конвертирует его в указанный ваш (новый_тип_данных).

Пример использования оператора static_cast для конвертации типа char в тип int:

```
#include <iostream>

int main()
{
    char ch{97};
    cout << ch << " " << ch << endl;

    int i{static_cast<int>(ch)};
    cout << ch << " " << i << endl;

    int ch{'5'};
    cout << ch << endl;

    return 0;
}
```

Результат выполнения программы:

```
a 97
97 97
5
```

Запомните, static_cast принимает (**выражение**) в качестве входных данных. Если мы используем переменную в (**выражение**), то эта переменная изменяет свой тип только в стейтменте с оператором static_cast. Процесс конвертации никак не влияет на исходную переменную с её значением в вышеприведенном примере, переменная ch остается переменной типа char с прежним значением, между является подтверждением последующий стейтмент с cout.

Также в static_cast нет никакой проверки по диапазону, так что если вы попытаетесь использовать числа, которые будут слишком большие или слишком маленькие для конвертируемого типа, то произойдёт **переполнение**.

Более подробно о static_cast вы еще поговорим на соответствующем уроке.

Ввод символов

Следующая программа просит пользователя ввести символ. Затем она выводит этот символ и его ASCII-код:

```
#include <iostream>

int main()
{
    cout << "Input a keyboard character: ";

    char ch;
    cin >> ch;
    cout << ch << " has ASCII code " << static_cast<int>(ch) << endl;

    return 0;
}
```

Результат выполнения программы:

```
Input a keyboard character: q
q has ASCII code 113
```

Обратите внимание, даже если cin позволит вам ввести несколько символов, переменная ch будет хранить только первый символ (именно он и помещается в переменную). Оставшаяся часть пользовательского ввода остается во входном буфере, который используют cin, и будет доступна для использования последующим вызовом cin.

Рассмотрим это всё на практике:

```
#include <iostream>

int main()
{
    cout << "Input a keyboard character: ";

    char ch;
    cout << "Input a keyboard character: ";// Спрашиваем, чтобы тот пользователь ввел abcd
    cin >> ch;
    cout << ch << " has ASCII code " << static_cast<int>(ch) << endl;

    char ch;
    cout << "Input a keyboard character: ";// Спрашиваем, чтобы тот пользователь ввел abcd
    cin >> ch;// ch = 'b'. Остальные буквы остались в буфере
    cout << ch << " has ASCII code " << static_cast<int>(ch) << endl;

    return 0;
}
```

Результат выполнения программы:

```
Input a keyboard character: abcd
a has ASCII code 97
b has ASCII code 98
```

Размер, диапазон и знак типа char

В языке C++ есть **управляющие символы** (или *«escape-последовательности»*). Они начинаются с бэкслеша ('\'), а затем следует определенная буква или цифра.

Наиболее распространенным управляющим символом в языке C++ является '\n', который обозначает символ новой строки:

```
#include <iostream>

int main()
{
    cout << "First line\nSecond line" << endl;

    return 0;
}
```

Результат:

```
First line
Second line
```

Еще одним часто используемым управляющим символом является '\t', который заменяет клавишу TAB, вставляя большой отступ:

```
#include <iostream>

int main()
{
    cout << "First part\tSecond part";

    return 0;
}
```

Результат:

```
First part      Second part
```

Таблица всех управляющих символов в языке C++:

Название	Символ	Значение
Предупреждение (alert)	\a	Предупреждение (звуковой сигнал)
Backspace	\b	Перемещение курсора на одну позицию назад
Formfeed	\f	Перемещение курсора к следующей логической странице
Символ новой строки (newline)	\n	Перемещение курсора на следующую строку
Возврат каретки (carriage return)	\r	Перемещение курсора в начало строки
Горизонтальный тэб (horizontal tab)	\t	Вставка горизонтального TAB
Вертикальный тэб (vertical tab)	\v	Вставка вертикального TAB
Одинарная кавычка	\'	Вставка одинарной кавычки (или апострофа)
Двойная кавычка	\"	Вставка двойной кавычки
Backslash	\\	Вставка обратной косой черты (backslash)
Вопросительный знак	\?	Вставка знака вопроса
Восьмеричное число	\number	Перевод числа из восьмеричной системы счисления в тип char
Шестнадцатеричное число	\xnumber	Перевод числа из шестнадцатеричной системы счисления в тип char

Рассмотрим пример в коде:

```
#include <iostream>

int main()
{
    cout << "\"This is quoted text\"";
    cout << "This string contains a single backslash \\" << endl;
    cout << "Bf in hex is char '\x4b\0'" << std::endl;

    return 0;
}
```

Результат выполнения программы:

```
"This is quoted text"
This string contains a single backslash \
Bf in hex is char 'K'
```

Что использовать: '\n' или std::endl?

Вы могли заметить, что в последнем примере мы использовали '\n' для перемещения курсора на следующую строку. Но мы могли бы использовать и std::endl. Какая между ними разница? Сейчас разберемся.

При использовании std::cout, данные для вывода могут помещаться в буфер, т.е. std::cout может не отправлять данные сразу же на вывод. Вместо этого он может остаться из-за себя на некоторое время (в целях улучшения производительности).

И '\n', и std::endl: оба перемещают курсор на следующую строку. Только std::endl: еще гарантирует, что все данные из буфера будут выведены, перед тем, как продолжить.

Так когда же использовать '\n', а когда std::endl?

- Используйте std::endl, когда нужно, чтобы ваши данные выводились сразу же (например, при записи в файл или при обновлении индикатора состояния какого-либо процесса. Обратите внимание, это может повлечь за собой незначительное снижение производительности, особенно если запись на устройство происходит медленно (например, запись файла на диск).
- Используйте '\n' во всех остальных случаях.

Другие символьные типы: wchar_t, char16_t и char32_t

Тип wchar_t действует практически во всех случаях (кроме тех, когда происходит взаимодействие с Windows API).

Так же, как и стандарт ASCII использует целые числа для представления символов английского языка, так и другие кодировки используют целые числа для представления символов других языков. Наиболее известный стандарт (после ASCII) — **Unicode**, который может хранить более 110 000 целых чисел для представления символов из разных языков.

Существует следующие кодировки Unicode:

- **UTF-32** — требует 32 бита для представления символа.
- **UTF-16** — требует 16 бит для представления символа.
- **UTF-8** — требует 8 бит для представления символа.

Типы char16_t и char32_t были добавлены в C++11 для поддержки 16-битных и 32-битных символов Unicode (8-битные символы и так поддерживаются типом char).

В чём разница между одинарными и двойными кавычками при использовании с символами?

Как вы уже знаете, символы всегда помещаются в одинарные кавычки (например, 'a', '+', '5'). Переменная типа char представляет только один символ (например, буква a, символ +, число 5). Следующий стейтмент не является корректным:

```
char ch{'Hello, world!'};
```

Текст, который находится в двойных кавычках, называется строкой (например, "Hello, world!"). **Строка** (тип **string**) — это набор последовательных символов.

Вы можете использовать литералы типа string в коде:

```
#include <iostream>
#include <string>

int main()
{
    cout << "Hello, world!" << endl;

    return 0;
}
```